

Randongeon — Guia de Estudo do Backend (Sabatina de POO)

Documento de estudo para a prova de laboratório de hoje à tarde. Foco: **orientação a objetos** aplicada ao backend do Randongeon (pacote `randongeon/jogo/`). O backend é puramente Python — não há framework de jogo; é um RPG de masmorra procedural. Como o professor costuma perguntar: * "O que é isso que vocês usaram? Para que serve?" * "Se eu executar essa parte, o que acontece? O que retorna?" * "Sempre vai ser assim?" O documento segue esse roteiro: **o que é** → **onde aparece no código** → **o que faz** → **quando o comportamento muda**.

0. Mapa geral do backend

randongeon/	
■■■■ main.py	← CLI (loop principal do jogo no terminal)
■■■■ jogo/	
■ ■■■■ entidades/	← "coisas" do jogo (POO pura)
■ ■ ■■■■ entidade.py	← Entidade (ABC) – base abstrata
■ ■ ■■■■ jogador.py	← Jogador
■ ■ ■■■■ inimigo.py	← Inimigo + Nosferatu, Golem, Banshee, Orc, Troll, CoracaoDaMasmorra
■ ■ ■■■■ item.py	← Item
■ ■ ■■■■ efeitos.py	← EfeitoStatus (ABC), Veneno, Fraqueza, EsquivaReduzida
■ ■ ■■■■ dom.py	← Dom (bruto, resistente, agil, sortudo, sanguessuga)
■ ■ ■■■■ loja.py	← Loja
■ ■■■■ sistemas/	← "regras / fluxo" do jogo
■ ■■■■ gerador.py	← GeradorSala – geração procedural
■ ■■■■ masmorra.py	← Masmorra – loop de andares, combate, fuga
■ ■■■■ persistencia.py	← serializar/desserializar save
■■■■ (api/)	← API FastAPI (caso o professor pergunte: ela chama o pacote jogo/; a lógica está aqui)

Pergunta provável: "Onde fica a lógica do jogo?" Resposta: nas **entidades** (estado + comportamento) e nos **sistemas** (regras de fluxo). A `main.py` só orquestra, e a `api/` é só um meio-de-campo HTTP.

1. Abstração + Classes Abstratas (ABC)

1.1 O que é

Uma **classe abstrata** define uma *interface* — um contrato — mas não pode ser instanciada. Ela diz: "qualquer coisa que seja deste tipo precisa ter estes métodos, mas não digo como eles são implementados".

Em Python, usamos o módulo `abc` (`from abc import ABC, abstractmethod`). A classe herda de `ABC` e marca os métodos "obrigatórios" com o decorator `@abstractmethod`.

1.2 Onde aparece no código

Duas classes-base claras:

- **Entidade** em `jogo/entidades/entidade.py`:

```
from abc import ABC, abstractmethod

class Entidade(ABC):
    def __init__(self, nome: str, hp: int) -> None:
        ...
```

```

        self.nome = nome
        self.hp_max = hp
        self.hp = hp
        self.efeitos: list = []

    def esta_vivo(self) -> bool:    # comportamento COMUM (não abstrato)
        return self.hp > 0

    def curar(self, quantidade: int) -> int:    # comportamento COMUM
        ...

    @abstractmethod
    def receber_dano(self, dano: int) -> int:
        """Cada subclasse aplica o dano do seu jeito."""
        raise NotImplementedError

```

- **EfeitoStatus** em `jogo/entidades/efeitos.py`:

```

class EfeitoStatus(ABC):
    tipo: str = "efeito"
    remove_ao_curar: bool = False

    def ao_iniciar_turno(self, portador) -> int: return 0
    def modifica_atk(self, atk: int) -> int: return atk
    def modifica_esquiva(self, esq: float) -> float: return esq

```

1.3 "Se eu executar `Entidade(...)` o que acontece?"

Resposta: **dá erro**. Como **Entidade** herda de **ABC** e tem um `@abstractmethod`, ela não pode ser instanciada:

```

>>> Entidade("Bug", 10)
TypeError: Can't instantiate abstract class Entidade ...

```

A classe existe só para forçar **Jogador** e **Inimigo** a fornecerem o método `receber_dano`.

1.4 Por que isso é útil aqui?

- O código que cuida de combate (em `Masmorra.resolver_combate`) trata

qualquer **Entidade** de forma uniforme: chama `esta_vivo()`, `receber_dano()`, `curar()`. Não importa se é o herói ou o bicho.

- Os efeitos de status (Veneno, Fraqueza, EsquivaReduzida) também são tratados

uniformemente via **EfeitoStatus**: a **Entidade** chama `ao_iniciar_turno()` e `modifica_atk()` sem saber qual é o efeito concreto.

1.5 "Sempre vai ser assim?"

Sim, **enquanto você chamar os hooks certos**. Se uma subclasse esquecer de sobrescrever `receber_dano`, o Python reclama no momento de instanciá-la:

```

class X(Entidade): pass
X("a", 1)    # TypeError – não implementou receber_dano

```

2. Herança

2.1 O que é

Uma subclasse **herda** atributos e métodos da classe-pai, podendo:

- reusar (`super().__init__`) para não duplicar validação,
- **sobrescrever** (override) para mudar o comportamento,
- **adicionar** coisas novas.

2.2 Onde aparece no código

Cadeia de heranças do projeto:

Entidade (ABC)	← base abstrata: nome, hp, esta_vivo, curar
■■■ Jogador	← jogador.py
■■■ Inimigo	← inimigo.py
■■■ Nosferatu	← vampiro (lifesteal)
■■■ GolemDePedra	← tanque com armadura
■■■ HordaDeGoblins	← bando (caso especial)
■■■ Banshee	← atordoa
■■■ Orc	← elite "esperto" (esquiva + fraqueza)
■■■ TrollDasCavernas	← tanque de HP (sem armadura)
■■■ CoracaoDaMasmorra	← boss de 2 fases
■■■ Goblin	← integrante de uma horda
EfeitoStatus (ABC)	← base de efeito de status
■■■ Veneno	← DoT 1/turno
■■■ Fraqueza	← -ATK por N turnos
■■■ EsquivaReduzida	← -ESQ por N turnos

2.3 Exemplo: Jogador herdando de Entidade

```
class Jogador(Entidade):
    def __init__(self, nome, hp=20, atk=5, xp=0, esq=0.3, moedas=0):
        # Valida nome/hp e define self.nome, self.hp, self.hp_max.
        super().__init__(nome, hp) # ← reaproveita o construtor do pai
        if atk <= 0: raise ValueError(...)
        self.atk = atk
        self.nivel = 1
        ...

    def receber_dano(self, dano: int) -> int: # ← implementação obrigatória
        if dano < 0: raise ValueError(...)
        dano_efetivo = min(dano, self.hp)
        self.hp -= dano_efetivo
        return dano_efetivo
```

O que **Jogador** ganha **de graça** da **Entidade**:

- **nome**, **hp_max**, **hp**, **efeitos** (atributos definidos no `__init__` da base);
- **esta_vivo()** e **curar()** (métodos concretos da base);
- **aplicar_efeito()**, **buscar_efeito()**, **remover_efeitos()**,

processar_efeitos_turno() (todo o sistema de status).

2.4 Exemplo: GolemDePedra herdando de Inimigo

```
class GolemDePedra(Inimigo):
    def __init__(self, bonus_hp=0, bonus_atk=0, bonus_armadura=0):
        super().__init__(
            nome="Golem de Pedra",
            hp=random.randint(15, 22) + bonus_hp,
            atk=random.randint(3, 5) + bonus_atk,
            dificuldade=2,
            xp=50, moedas=random.randint(10, 15),
            modificador_fuga=0.40,
            absorcao_dano=3 + bonus_armadura, # ← armadura do tanque
```

```

        tipo_especial="golem",
        chance_miss=0.10, chance_drop=0.20,
    )

    def tabela_loot(self) -> list: # ← override
        return LOOT_GOLEM

```

`GolemDePedra` herda `atacar()`, `receber_dano()` (com armadura), `curar()`, `rolar_dano()` não — `rolar_dano` é do `Jogador`. O `Golem` ainda sobrescreve `tabela_loot()` (polimorfismo, ver §3).

2.5 "Se eu instanciar `Jogador('a', hp=0)`, o que acontece?"

`Entidade.__init__` valida:

```

if hp <= 0:
    raise ValueError("HP inicial deve ser maior que zero.")

```

Resposta: levanta `ValueError`. O mesmo vale para `Inimigo`, `Item`, `EfeitoStatus`, `Loja`, `GeradorSala` — todos validam no construtor.

2.6 "Sempre vai cair nas validações?"

Sim, **a menos que você construa o objeto de outro jeito** (por exemplo, desserializando de um JSON com valores inválidos, ou sobrescrevendo `__init__` sem chamar `super().__init__`). É por isso que a função `desserializar_jogador` em `persistencia.py` faz o seguinte:

```

j = Jogador(nome=..., hp=int(data["hp_max"]), ...)
j.hp = int(data["hp"]) # ← atribuição direta (passa pela validação
                       # do construtor pelo HP máximo; depois
                       # rebaixa o hp atual sem validação).

```

É um caso onde a validação é propositalmente "frouxa" para restaurar o save (ex.: você morreu com HP baixo e quer carregar de novo).

3. Polimorfismo

3.1 O que é

Mesma chamada, **comportamento diferente** dependendo do tipo concreto. Em POO, o polimorfismo aparece de três formas:

1. **Override de método**: subclasse reescreve um método do pai.
2. **Duck typing** (Python): se o objeto tem o método, funciona.
3. **Hook abstrato**: a base define o que chamar; a subclasse define como.

3.2 Onde aparece no código (3 exemplos fortes)

3.2.1 `receber_dano()` — `Jogador` vs `Inimigo`

```

# Em Jogador (sem armadura):
def receber_dano(self, dano: int) -> int:
    dano_efetivo = min(dano, self.hp)
    self.hp -= dano_efetivo
    return dano_efetivo

# Em Inimigo (desconta armadura):
def receber_dano(self, dano: int) -> int:
    dano_apos_absorcao = max(0, dano - self.absorcao_dano)
    dano_efetivo = min(dano_apos_absorcao, self.hp)

```

```
self.hp -= dano_efetivo
return dano_efetivo
```

Quem chama (**masmorra.py**) faz a mesma coisa nos dois:

```
dano = inimigo.receber_dano(dano_base)      # Inimigo: desconta armadura
# e antes:
dano = alvo.receber_dano(self.atk)         # Jogador: não desconta nada
```

O que o professor pode perguntar: "Se o **alvo** for um Jogador, ele perde **self.atk** de HP?" — Resposta: depende, o **min(dano, self.hp)** impede que o HP fique negativo, e o **Jogador** não subtrai armadura porque não tem **absorcao_dano**. O método é polimórfico.

3.2.2 **tabela_loot()** — *pool por tipo de inimigo*

```
class Inimigo(Entidade):
    def tabela_loot(self) -> list:      # pool padrão
        return LOOT_PADRAO

class GolemDePedra(Inimigo):
    def tabela_loot(self) -> list:      # pool temático
        return LOOT_GOLEM

class Banshee(Inimigo):
    def tabela_loot(self) -> list:
        return LOOT_BANSHEE
```

E o sistema de loot não sabe o tipo concreto:

```
# masmorra.py — _rolar_loot
pool = inimigo.tabela_loot() # polimorfismo: cada inimigo decide
return random.choice(pool) if random.random() < chance else None
```

O que o professor pode perguntar: "Se eu matar um Golem, qual o loot?" Resposta: sai do **LOOT_GOLEM** (Fragmento de Pedra ou Núcleo de Pedra). Se eu matar um Esqueleto (Inimigo "puro"), sai do **LOOT_PADRAO**.

3.2.3 **EfeitoStatus** — *hooks de combate*

```
class EfeitoStatus(ABC):
    def ao_iniciar_turno(self, portador) -> int: return 0 # default
    def modifica_atk(self, atk: int) -> int: return atk # default
    def modifica_esquiva(self, esq: float) -> float: return esq # default

class Veneno(EfeitoStatus):
    tipo = "veneno"
    def ao_iniciar_turno(self, portador) -> int:
        return portador.receber_dano(self.DANO) # ← override

class Fraqueza(EfeitoStatus):
    tipo = "fraqueza"
    def modifica_atk(self, atk: int) -> int:
        return max(1, atk - self.reducao) # ← override
```

E o **Jogador** chama todos os hooks sem **if**:

```
# jogador.py — atk_efetivo()
for efeito in self.efeitos:
    atk = efeito.modifica_atk(atk) # Veneno não faz nada; Fraqueza reduz
return atk
```

Esse padrão é o que os slides da disciplina chamam de **Template Method**: a base define o esqueleto (**processar_efeitos_turno**, **atk_efetivo**); a subclasse preenche os detalhes.

3.2.4 **tentar_renascer()** — *Template Method no boss*

```

class Inimigo(Entidade):
    def tentar_renascer(self) -> bool:
        return False          # default: morre de vez

class CoracaoDaMasmorra(Inimigo):
    def tentar_renascer(self) -> bool:
        if self._ja_renasceu or self.esta_vivo():
            return False
        self._ja_renasceu = True
        self.curar(max(1, round(self.hp_max * CORACAO_CURA_RENASCIMENTO)))
        self.atk = max(self.atk + 1, round(self.atk * CORACAO_FURIA_ATK_MULT))
        return True

```

O fluxo de morte é igual para todos os inimigos:

```

# masmorra.py
if inimigo.tentar_renascer():      # polimorfismo
    print(MENSAGEM_RENASCIMENTO)
    continue                       # recomeça o combate
break                             # morreu de vez

```

"E o Bando de Goblins?" Ele **não** herda de Inimigo (ver §6 — composição). O tratamento é separado: a **Masmorra** lida com a fila de goblins manualmente (no combate, é resolvido um goblin por vez; o resto está no estado da sessão).

3.3 "Sempre vai cair no mesmo branch?"

Em Python, **o método que roda é o do tipo concreto do objeto**, não o da variável que aponta para ele. Se você tem:

```

def cair_no_lava(inimigo: Inimigo):
    return inimigo.tentar_renascer()

cair_no_lava(Inimigo("X", 1, 1, 1, 1, 1))  # → False
cair_no_lava(CoracaoDaMasmorra(...))      # → True (1ª vez), False (2ª)

```

4. Encapsulamento

4.1 O que é

Esconder os detalhes internos de um objeto e expor só uma **interface pública** (o que outros módulos podem usar). Em Python não há **private** de verdade — usamos **convenção**:

- **nome** → público.
- **_nome** → "protegido" (uso interno, não chamar de fora).
- **__nome** → "privado" (name mangling — vira **_Classe__nome**).

4.2 Onde aparece no código

4.2.1 Atributos **_protegidos**

```

# jogador.py
self._ja_renasceu = False  # (no CoracaoDaMasmorra) — só ele mexe.

# entidade.py
def _atualizar_nivel(self) -> int:    # ← método "privado por convenção"
    ...

```

O que o professor pode perguntar: "Por que **_atualizar_nivel** começa com underline?" — Resposta: é convenção para indicar que é um detalhe interno. Quem chama de fora usa

`jogador.ganhar_xp(50)`, que internamente chama `_atualizar_nivel`. Não é para ser chamado diretamente.

4.2.2 `super().__init__()` — delegar a construção para a base

```
class Jogador(Entidade):
    def __init__(self, nome, hp=20, atk=5, ...):
        super().__init__(nome, hp) # ← validações e atributos vão pra base
        if atk <= 0: raise ValueError(...)
        self.atk = atk
```

4.2.3 `@property` — getter calculado

```
# jogador.py
@property
def pontuacao(self) -> int:
    return self.xp + (self.nivel - 1) * 50 + self.moedas

# uso externo:
score = jogador.pontuacao # calculado na hora, sem parênteses
```

Vantagem: parece atributo, mas é calculado. Não dá para setar de fora (`jogador.pontuacao = 999` → `AttributeError`). Encapsula a fórmula.

4.2.4 Validações no construtor

Todas as classes validam entradas e levantam `ValueError` se algo estiver errado. Isso **encapsula a invariante** do objeto: depois de construído, o objeto está em estado válido.

```
# Inimigo
if not (0.0 <= chance_atordoar <= 1.0):
    raise ValueError(...)
```

"Se eu criar um inimigo com `chance_atordoar=2`, o que acontece?" Resposta: `ValueError` na hora de instanciar. Não dá para existir um inimigo com 200% de chance de atordoar.

4.2.5 Métodos que delegam (ex.: `aumenta_esq`)

```
# item.py – Item não escreve direto em jogador.esq
esq_recuperada = jogador.aumenta_esq(self.bonus_esq)

# jogador.py
def aumenta_esq(self, quantidade: int) -> int:
    esq_antes = self.esq
    self.esq = min(self.esq_max, self.esq + quantidade) # respeita teto
    return self.esq - esq_antes
```

Por que delegar? Para respeitar o teto (`esq_max`). Se `Item` atribuísse direto (`jogador.esq += 1.5`), poderia estourar. Encapsulamento: o objeto é o **dono** do seu próprio estado.

4.3 "Sempre vai dar certo?"

- Validações só rodam **na construção**. Se você fizer

`jogador.hp = -5` direto, vai aceitar (atributo público, sem setter).

- Em Python, **ninguém impede** o código externo de mexer em `_atributos`

"privados". É convenção. Se a regra é importante, ela precisa estar em um **método** (como `aumenta_esq`).

5. Atributos de Instância vs de Classe

5.1 O que é

- **Atributo de instância:** vive no objeto (`self.x`). Cada instância tem o seu.
- **Atributo de classe:** vive na classe (`Classe.X`). Compartilhado por todas as instâncias. Usado para constantes e regras comuns.

5.2 Onde aparece no código

```
# jogador.py – constantes de balanceamento
class Jogador(Entidade):
    ATK_POR_NIVEL      = 2
    HP_POR_NIVEL       = 12
    XP_BASE_NIVEL      = 10
    ESQ_MAXIMA         = 0.6
    CHANCE_CRITICO_BASE = 0.10
    MULTIPLICADOR_CRITICO = 1.5
    CURA_NIVEL_FRACAO  = 0.60

# inimigo.py – constantes de balanceamento
ESCALA_HP_POR_ANDAR    = 1.8
ELITE_HP_MULTIPLICADOR = 1.4
CHANCE_VENENO          = 0.08
```

O que o professor pode perguntar: "Por que não usar variável global?" Resposta: ficam **junto da classe** que as usa (alta coesão). E são acessíveis como `Jogador.ATK_POR_NIVEL`, o que deixa a intenção clara em `self.atk += self.ATK_POR_NIVEL`.

5.3 "Se eu mudar `Jogador.ATK_POR_NIVEL` em tempo de execução, o que acontece?"

Resposta: **muda para todas as instâncias** (e para as futuras também) — esse é o comportamento padrão de atributo de classe. Por isso constantes não são "alteradas" no fluxo do jogo; são usadas como base de cálculo:

```
self.atk += self.ATK_POR_NIVEL    # lê, não escreve
```

6. Composição (vs Herança)

6.1 O que é

"Tem um" em vez de "é um". Em vez de herdar, o objeto guarda uma referência a outro.

6.2 Onde aparece no código

O caso mais claro é o **Bando de Goblins**:

```
class BandoDeGoblins:
    """
    Composição: um Bando NÃO é um Inimigo —
    ele TEM vários Goblins (relação "tem um").
    """
    TAMANHO = 3
    def __init__(self) -> None:
        hp      = random.randint(4, 7)
        atk      = random.randint(1, 2)
        ...
        self.goblins = [
```



```

        Goblin("Bando de Goblins", hp=hp, atk=atk, ...)
    for _ in range(self.TAMANHO)
]

```

E o **Jogador** tem um inventário, tem uma lista de efeitos, tem um dom:

```

self.inventario: list = [] # tem vários Itens
self.efeitos: list = [] # tem vários EfeitoStatus
self.dom: str | None = None # tem um Dom (ou nenhum)

```

"Herança ou composição?" Regra prática: * **É um** → herança (**Banshee** é um **Inimigo**). * **Tem um(s)** → composição (**BandoDeGoblins** tem **Goblins**; **Jogador** tem **Items**).

6.3 Por que composição aqui?

Se o **BandoDeGoblins** herdasse de **Inimigo**, ele teria UM único **hp/atk** — mas a ideia é que o herói enfrente 3 goblins **em sequência**. Então a coleção é o modelo certo.

7. Métodos estáticos, de classe e **getattr**

7.1 **@staticmethod** — função "dentro" da classe

```

# inimigo.py
@staticmethod
def gerar(andar: int = 1) -> "Inimigo":
    ...

```

O que é: um método que **não** recebe **self** nem **cls**. É só uma função agrupada na classe por organização. Pode ser chamado como **Inimigo.gerar(5)** ou **obj.gerar(5)** — funciona igual. "Se eu chamar **Inimigo.gerar(5)**, o que retorna?" Resposta: uma instância aleatória de **Inimigo** (ou de uma subclasse como **GolemDePedra**, **Banshee**...) escalada para o andar 5.

7.2 **getattr(obj, nome, default)** — *duck typing defensivo*

Aparece várias vezes — é um padrão de "se o objeto tiver esse atributo, use; senão, use um default":

```

# inimigo.py – atacar()
miss = self.chance_miss + getattr(alvo, "evasao_passiva", 0.0)

# masmorra.py – varios pontos
absorcao = getattr(inimigo, 'absorcao_dano', 0)
tipo = getattr(inimigo, 'tipo_especial', None)

```

Por que não **alvo.evasao_passiva** direto? Porque, em testes unitários, mocks podem não ter esse atributo. **getattr(..., 0.0)** garante que sempre funcione. "Sempre vai usar o default?" Não: se o objeto tem o atributo, usa o valor dele. Se não tem, usa o default.

8. **@property** — outro uso: veneno_turnos e envenenado

```

# jogador.py
@property
def veneno_turnos(self) -> int:
    """Turnos de veneno restantes – derivado do efeito ativo."""
    efeito = self.buscar_efeito("veneno")
    return efeito.turnos if efeito else 0

@property
def envenenado(self) -> bool:

```

```
"""True se o jogador ainda tem turnos de veneno ativos."""
return self.veneno_turnos > 0
```

"Se o jogador não estiver envenenado, o que retorna?" `veneno_turnos` retorna 0, `envenenado` retorna `False`. "E se o veneno já durou 0 turnos?" O `buscar_efeito` só devolve efeito com `ativo() == True` (turnos > 0), então retorna 0.

9. Coleções importantes (e o que nelas tem)

9.1 Jogador.efeitos — lista de EfeitoStatus

- Suporta vários efeitos diferentes ao mesmo tempo (Veneno + Fraqueza, por exemplo).

- A `Entidade` processa todos a cada turno via `processar_efeitos_turno()`.

- Efeitos do **mesmo tipo** não empilham — `aplicar_efeito()` renova a duração para a maior:

```
# entidade.py
def aplicar_efeito(self, efeito) -> None:
    existente = self.buscar_efeito(efeito.tipo)
    if existente is not None:
        existente.turnos = max(existente.turnos, efeito.turnos)
    else:
        self.efeitos.append(efeito)
```

```
"""Se o jogador já está envenenado e leva outra picada, ele fica com 2 * VENENO_DURACAO
turnos?""" Não. Renova para o MAIOR valor, e o Veneno cap em Jogador.VENENO_DURACAO:
```

```
# jogador.py — envenenar
self.aplicar_efeito(Veneno(min(turnos, self.VENENO_DURACAO)))
```

9.2 Jogador.inventario — lista de Item

Operações:

- `adicionar_item(item)` — append.
- `usar_item(indice)` — pop e aplica `item.usar(jogador)`. Retorna o

`dict` que `Item.usar` devolve.

- `inventario_resumo()` — projeção serializável (nome + bônus).

"Se eu chamar `usar_item(99)`, o que acontece?" Levanta `IndexError` (validação no início do método).

9.3 Loja.ofertas — lista de dicts (item + preço)

```
self.estoque = [
    {"item": Item("Grande Poção de Força", bonus_atk=2), "preco": 15},
    ...
]
self.ofertas = random.sample(self.estoque, 2) # 2 ofertas aleatórias
```

`comprar(indice, jogador):`

1. Valida índice.

2. Verifica `jogador.moedas >= preco`.
3. Debita moedas, adiciona item ao inventário, remove a oferta.
4. Retorna `{"sucesso": bool, "mensagem": str}`.

"Se o jogador não tiver moedas?" Retorna `{"sucesso": False, "mensagem": "Moedas insuficientes."}`. O item **não** é adicionado.

10. Laços importantes

10.1 `Masmorra.resolver_combate` — *duplo while*

```
while True:                                # ← laço externo (boss renasce)
    while self.jogador.esta_vivo() and inimigo.esta_vivo():
        # turno do jogador (com chance de miss, esquiva do inimigo)
        ...
        # turno do inimigo
        relatorio = inimigo.atacar(self.jogador)
        ...
    if not self.jogador.esta_vivo():
        return "derrota"
    if inimigo.tentar_renascer():            # ← boss 2ª fase
        continue
    break
```

"Por que dois **while**?" O externo é para o caso da 2ª fase do boss (renasce uma vez). O interno é o combate em si. Para inimigos normais, `tentar_renascer()` devolve **False** e o **break** cai.

10.2 `Jogador._atualizar_nivel` — *pode subir vários de uma vez*

```
while self.xp >= self.xp_para_proximo_nivel():
    self.nivel += 1
    self.atk    += self.ATK_POR_NIVEL
    ...
    niveis_ganhos += 1
return niveis_ganhos
```

"Se eu ganhar 10.000 XP de uma vez, sobe um nível só?" Não. Sobe todos os níveis que o XP acumulado permitir (loop).

11. Curvas e fórmulas (memorize as duas)

11.1 *XP para próximo nível (triangular)*

```
def xp_para_proximo_nivel(self) -> int:
    return self.XP_BASE_NIVEL * self.nivel * (self.nivel + 1)
```

- Nível 1→2: $10 \cdot 1 \cdot 2 = 20$ XP
- Nível 2→3: $10 \cdot 2 \cdot 3 = 60$ XP
- Nível 3→4: $10 \cdot 3 \cdot 4 = 120$ XP

Por que triangular? Primeiros níveis chegam rápido (sensação de progresso); os altos exigem cada vez mais XP.

11.2 *Escala de inimigo por andar*

```
bonus_hp = round(andar * 1.8)
```

```
bonus_atk    = andar // 5
bonus_moedas = andar // 2
```

"No andar 10, quanto mais forte é o comum?" HP + 18, ATK + 2, moedas + 5. (O boss escala com fórmula própria.)

12. Fluxo de uma run (perfeito para "se eu rodar, o que acontece?")

```
main()
■ ■ iniciar_run(historico)
    ■ ■ GeradorSala()                ← cria o gerador
    ■ ■ Masmorra(jogador=None)      ← cria a masmorra vazia
    ■ ■ masmorra.mostrar_lore()      ← poesia opcional
    ■ ■ input("nome")                ← nome do herói
    ■ ■ Jogador(nome, hp=20, atk=5)
    ■ ■ masmorra.jogador = jogador
    ■ ■ masmorra.jogar()             ← loop principal
        ■ ■ while jogador.vivo and not desistiu:
            ■ ■ menu() → escolha
            ■ ■ "1" → avancar()
            ■ ■     ■ ■ andar += 1
            ■ ■     ■ ■ se andar % 5 == 0 → gerar_boss() → _combate_interativo
            ■ ■     ■ ■ senão:
            ■ ■         ■ ■ gerador.gerar_sala(andar) → ('loja'|'item'|'inimigo', ...)
            ■ ■         ■ ■ loja → Loja().menu()
            ■ ■         ■ ■ item → aplicar_item(item)
            ■ ■         ■ ■ inimigo → _combate_interativo(inimigo)
            ■ ■ "2" → mostrar_status()
            ■ ■ "3" → desistiu = True
```

"O que `Masmorra()` com `jogador=None` faz?" Funciona. O atributo `self.jogador` é setado depois em `iniciar_run`. Isso é proposital: precisamos rodar `mostrar_lore()` antes de pedir o nome.

13. Padrão "Template Method" no combate

O esqueleto está em `Masmorra.resolver_combate`. O detalhamento está em `Inimigo.atacar()`. Vantagens:

1. **Sem `if tipo == ...`** no laço de combate. Polimorfismo.
2. **Cada inimigo é dono das suas mecânicas:** o Nosferatu sabe roubar vida, a

Banshee sabe atordoar, o Orc sabe aplicar fraqueza. Adicionar um novo inimigo = criar uma subclasse, sem mexer na `Masmorra`.

```
# masmorra.py
relatorio = inimigo.atacar(self.jogador)
if relatorio["envenenou"]: self.jogador.envenenar()
if relatorio.get("fraqueza"): self.jogador.aplicar_efeito(Fraqueza(2))
```

A Masmorra só **reage** ao relatório. Quem decide "se vai envenenar" é o inimigo, no `atacar()`.

14. Pontos que costumam cair em prova

14.1 Diferença Jogador.receber_dano vs Inimigo.receber_dano

Aspecto	Jogador	Inimigo
Armadura	Não desconta	Desconta <code>absorcao_dano</code>
HP nunca negativo	Sim (<code>min(dano, hp)</code>)	Sim (<code>min(dano_apos, hp)</code>)
Retorna	Dano efetivo	Dano efetivo
Validation	<code>dano < 0 → ValueError</code>	<code>dano < 0 → ValueError</code>

14.2 O que é cada tipo_especial?

```
indicadores = {
    "nosferatu": " ■ [Regeneração 20%]",
    "golem":     " ■ [Armadura: 2]",
    "horda":     " ■ [Horda]",
    "banshee":   " ■ [Atordoamento 30%]",
}
```

Esses valores **não** somam — cada inimigo é instanciado com **um** tipo (configurado no `super().__init__(...)`).

14.3 Ordem de resolução de uma rodada de combate

1. Verifica se jogador está **atordoado** (perde o turno).
2. Senão: `chance_miss` do jogador (10%). Se acertou e o inimigo não

desviou, aplica dano.

1. Aplica lifesteal do jogador (Sanguessuga).
2. Processa efeitos de status do jogador (Veneno, etc.).
3. Inimigo ataca: relatório com ``dano / errou / curou / atordoou /`

`envenenou / fraqueza / esquiva_reduzida / subiu_atk``.

1. Aplica cada debuff do relatório no jogador.

14.4 Por que Inimigo.atacar() retorna um dict e não altera o estado do jogador sozinho?

Para **separar responsabilidades**:

- O inimigo calcula o que **ele** causou (dano, lifesteal, atordoamento, etc.).
- A Masmorra (ou a API) **aplica** os efeitos no jogador e **gera a mensagem**

para o usuário.

Isso permite:

- A CLI narrar com `print(...)`.
- A API devolver o `dict` no JSON.
- Os testes unitários conferirem o relatório.

14.5 Como a loja sorteia as ofertas?

```
self.ofertas = random.sample(self.estoque, 2)
```

`random.sample` é **sem reposição**: as 2 ofertas são **distintas**. Se o estoque tiver 4 itens, há 6 combinações possíveis ($4 \cdot 3/2$).

15. Glossário rápido

- **ABC** — *Abstract Base Class*. Classe que define contrato, não pode ser instanciada.

- **Override** — reescrever um método herdado.
- **Hook** — método vazio (no-op) na base, sobrescrito pela subclasse.
- **Polimorfismo** — mesma chamada, comportamento diferente.
- **Composição** — "tem um" (vs herança: "é um").
- **Encapsulamento** — esconder detalhes, expor interface.
- **Atributo de classe** — compartilhado, vive na classe.
- **@property** — método que se acessa como atributo (getter calculado).
- **@staticmethod** — função dentro da classe, sem `self/cls`.
- **@classmethod** — recebe a classe como 1º arg (`cls`). Não usamos no

projeto, mas vale conhecer.

- `getattr(obj, "x", default)` — `obj.x` com fallback.
-

16. Perguntas-modelo (responder em voz alta treinando)

1. O que é **Entidade** e por que é abstrata?

É a classe base de **Jogador** e **Inimigo**. É abstrata porque tem um método abstrato (`receber_dano`) — não faria sentido instanciar uma "entidade genérica" sem saber como ela recebe dano.

1. Se eu criar `Jogador("Ana", hp=0)`, o que acontece?

`ValueError("HP inicial deve ser maior que zero.")` (vem da **Entidade**).

1. Se o herói atacar um Golem com ATK 10, quanto de dano o Golem sofre?

`max(0, 10 - absorcao_dano)`. Se `absorcao_dano = 3`, o Golem sofre 7. (E o `Inimigo.receber_dano` ainda limita a `self.hp` para não ficar negativo.)

1. Qual a diferença entre `Inimigo.atacar` e `Masmorra.resolver_combate`?

`atacar` é UM turno do inimigo (devolve um relatório). `resolver_combate` é o laço completo (jogador + inimigo, com mensagens).

1. Como o boss final revive?

`Masmorra.resolver_combate` chama `inimigo.tentar_renascer()` após o HP zerar. Para o `CoracaoDaMasmorra`, isso cura 50% do `hp_max` e aumenta `atk` em 25% — apenas uma vez (controlado por `_ja_renasceu`).

1. Por que o loot é decidido no inimigo, não na masmorra?

Polimorfismo: cada inimigo sobrescreve `tabela_loot()`. A masmorra só chama `inimigo.tabela_loot()`. Adicionar um novo inimigo com loot próprio é trivial (sem mexer na

masmorra).

1. Onde mora o veneno no jogador?

Em `Jogador.efeitos` (lista de `EfeitoStatus`). O `veneno_turnos` é uma `@property` que conta os turnos do efeito "veneno" ativo.

1. "Sempre vai acontecer assim?" — exemplos:

- `receber_dano` no `Inimigo` **sempre** desconta armadura? Sim, *desde que* a subclasse não sobrescreva. (Hoje nenhuma sobrescreve.)

- `Item.usar` **sempre** cura e purga veneno se tiver `bonus_hp`? Sim — o método é o mesmo, sem override de subclasse.

- `Jogador.ganhar_xp` **sempre** sobe de nível? Só se o XP acumulado passar do limiar da curva triangular; senão, retorna 0.

17. TL;DR (1 página)

- O backend está em `randoneon/jogo/` (entidades + sistemas). A `main.py` e a API só orquestram.

- Pilares de POO:

- **Abstração** → `Entidade` (ABC) e `EfeitoStatus` (ABC).

- **Herança** → `Jogador(Entidade)`, `Inimigo(Entidade)`, e a árvore de `Banshee/Nosferatu/Golem/Orc/Troll/CoracaoDaMasmorra/Inimigo`.

- **Polimorfismo** → `receber_dano` (Jogador vs Inimigo), `tabela_loot()`, `tentar_renascer()`, hooks de `EfeitoStatus`.

- **Encapsulamento** → validações no `__init__`, `@property` (pontuação, `veneno_turnos`), métodos que delegam (`umenta_esq`), `_protegidos` por convenção.

- **Composição** → `BandoDeGoblins` (tem `Goblins`), `Jogador` (tem `inventario`, `efeitos`, `dom`).

- **Atributos de classe** → constantes de balanceamento (`Jogador.ATK_POR_NIVEL`, `ESCALA_HP_POR_ANDAR`).

- **Template Method** → `Masmorra.resolver_combate` (esqueleto) + `Inimigo.atacar` (detalhe). `Entidade.processar_efeitos_turno` (esqueleto)

- `EfeitoStatus.ao_iniciar_turno` (detalhe).

- A `Masmorra` não usa `if tipo == ...` para resolver o combate — usa polimorfismo puro, centralizado em `Inimigo.atacar()`.

- O `Jogador.efeitos` é a lista canônica de debuffs (Veneno, Fraqueza, EsquivaReduzida). Cada efeito é um objeto; a Entidade chama os hooks.

- Veneno: o jogador tem o efeito, a `Entidade` processa, o `tick_veneno()`

é um atalho para `processar_efeitos_turno()` (mantido por compatibilidade com testes/laços antigos).

- O boss final renasce UMA vez: `tentar_renascer` na subclasse

`CoracaoDaMasmorra` cuida disso, controlado por `_ja_renasceu`.

- A loja sorteia 2 ofertas distintas de um catálogo de 4 (`random.sample`).

- A serialização de save guarda versão + jogador + andar + modo, e rejeita saves de versão superior.